Instituicao de Ensino Demo

Curso: Engenharia de Software

3o Semestre

Material de Estudo

Engenharia de Software I

---

1. INTRODUCAO

A Engenharia de Software aplica principios de engenharia ao desenvolvimento de software, buscando produzir sistemas de alta qualidade dentro do prazo e orcamento. Esta disciplina ensina processos, tecnicas e ferramentas para planejar, projetar, implementar e manter software de forma sistematica. E a base para trabalhar em equipes e projetos reais.

Ementa: Processos de desenvolvimento de software. Engenharia de requisitos. Modelagem UML. Ciclo de vida do software. Metodologias tradicionais (cascata, espiral). Qualidade de software.

2. CONCEITOS FUNDAMENTAIS

A seguir, apresentamos os conceitos-chave desta disciplina com explicacoes didaticas e exemplos praticos para facilitar o aprendizado.

2.1 Processos de Software e Ciclo de Vida

Um processo de software define as atividades necessarias para desenvolver um sistema. O modelo cascata segue fases sequenciais (requisitos, design, implementacao, teste). O modelo incremental entrega o software em partes. O modelo espiral combina iteracao com analise de riscos. A escolha do processo depende do projeto e da equipe.

Exemplo:

```
Modelo Cascata:
1. Requisitos -> 2. Design -> 3. Implementacao
-> 4. Testes -> 5. Manutencao

Vantagem: simples e bem documentado
Desvantagem: dificil voltar a fases anteriores

Modelo Incremental:
Incremento 1: funcionalidades basicas
Incremento 2: funcionalidades intermediarias
Incremento 3: funcionalidades avancadas
```

2.2 Engenharia de Requisitos

Requisitos definem O QUE o sistema deve fazer (funcionais) e COMO deve se comportar (nao-funcionais: performance, seguranca, usabilidade). A elicitacao coleta requisitos com stakeholders. A especificacao documenta em formato padrao (SRS). Requisitos mal definidos sao a principal causa de falha em projetos de software.

Exemplo:

```
Requisito funcional:
"O sistema deve permitir que o aluno consulte suas notas por disciplina."

Requisito nao-funcional:
"O tempo de resposta para consultas deve ser inferior a 2 segundos."
"O sistema deve suportar 1000 usuarios simultaneos."
```

3. RESUMO DOS PRINCIPAIS TOPICOS

O conteudo programatico da disciplina esta organizado nas seguintes unidades:

- Unidade 1: Introducao a engenharia de software - conceitos e importancia
- Unidade 2: Processos de software - cascata, incremental, espiral
- Unidade 3: Engenharia de requisitos - elicitacao e especificacao
- Unidade 4: Modelagem UML - diagramas de casos de uso
- Unidade 5: Modelagem UML - diagramas de classes e sequencia
- Unidade 6: Projeto de software - arquitetura e design
- Unidade 7: Documentacao de software - SRS, SDD
- Unidade 8: Qualidade de software - metricas e padroes (ISO 25010)

Cada unidade aborda um aspecto fundamental da disciplina, construindo progressivamente o conhecimento necessario para dominar o tema. Recomenda-se estudar as unidades na ordem apresentada.

4. EXERCICIOS DE FIXACAO

Resolva os exercicios abaixo para verificar seu entendimento dos conceitos apresentados.

Questao 1 (Multipla Escolha)

Qual modelo de processo segue fases estritamente sequenciais?

a) Espiral
b) Cascata
c) Agil
d) Incremental

Questao 2 (Dissertativa)

Diferencie requisitos funcionais de nao-funcionais. De 2 exemplos de cada para um sistema de e-commerce.

Questao 3 (Multipla Escolha)

Qual diagrama UML representa a interacao entre usuario e sistema?

a) Diagrama de classes
b) Diagrama de sequencia
c) Diagrama de casos de uso
d) Diagrama de atividades

5. DICAS DE ESTUDO E RECURSOS ADICIONAIS

Para maximizar seu aprendizado nesta disciplina, siga estas recomendacoes:

- Pratique modelagem UML com ferramentas como draw.io ou StarUML.
- Escreva requisitos para projetos pessoais antes de comecar a programar.
- Estude casos reais de falhas em projetos de software.
- Leia o livro do Sommerville — e a referencia classica da area.

Bibliografia Recomendada:

- SOMMERVILLE, I. Engenharia de Software. Pearson, 2019.
- PRESSMAN, R. Engenharia de Software: Uma Abordagem Profissional. McGraw-Hill, 2021.
- LARMAN, C. Utilizando UML e Padroes. Bookman, 2007.

6. GLOSSARIO DE TERMOS IMPORTANTES

Consulte este glossario sempre que encontrar um termo desconhecido durante seus estudos.

- **Requisito funcional**: o que o sistema deve fazer.
- **Requisito nao-funcional**: como o sistema deve se comportar (performance, seguranca).
- **UML**: Unified Modeling Language — linguagem de modelagem visual.
- **SRS**: Software Requirements Specification.
- **Stakeholder**: qualquer pessoa interessada ou afetada pelo sistema.
- **Ciclo de vida**: fases pelas quais o software passa, do inicio ao fim.